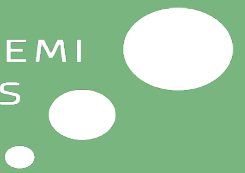
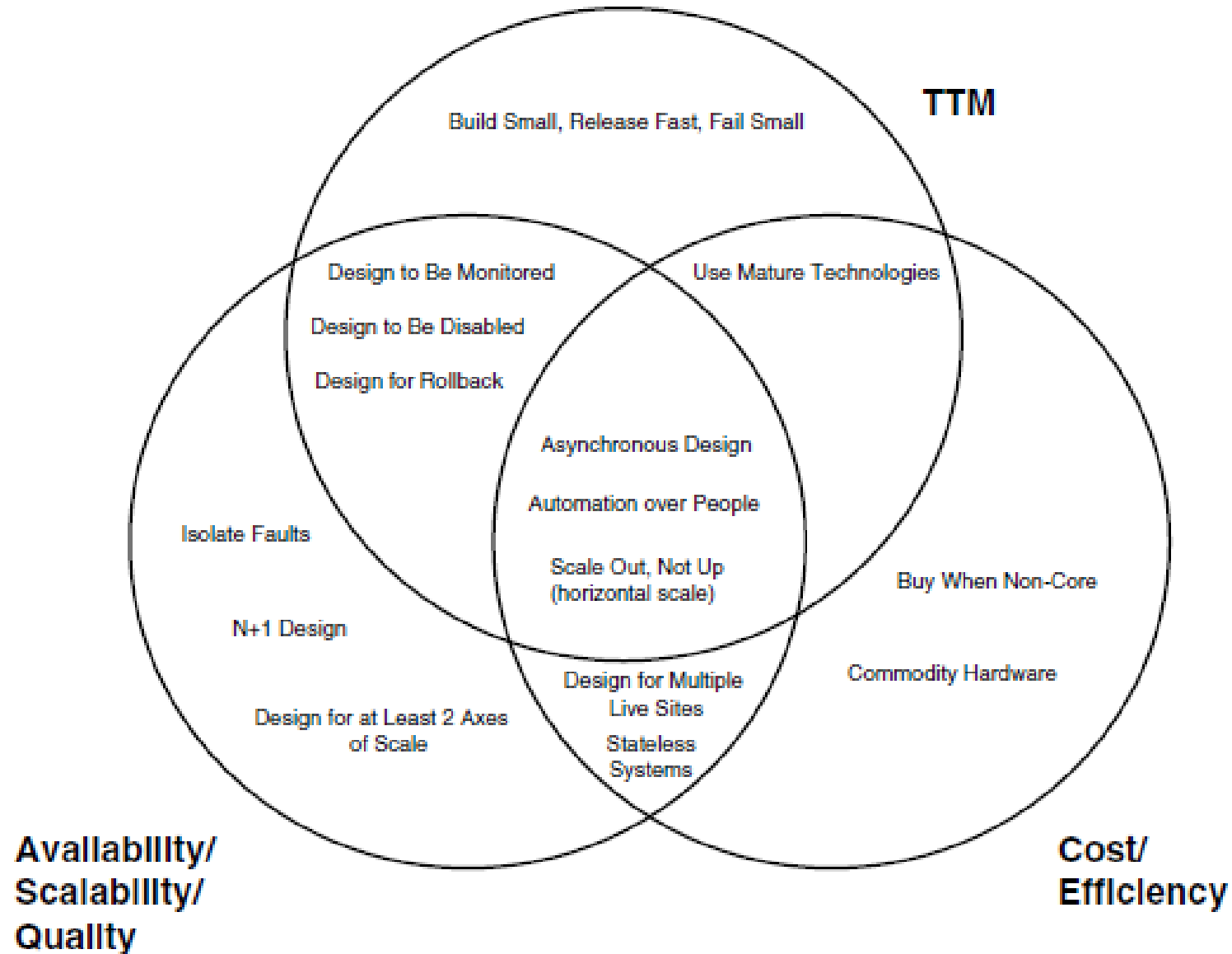


Deploymentmiljøer



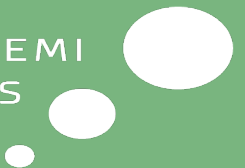
Arkitekt Principper i Operations



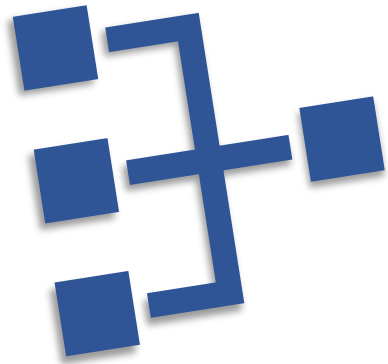
CI/CD Miljøoversigt

Miljø / Formål	Typiske aktiviteter	Ansvar
BUILD At kompilere og bygge applikationen fra kildekoden, så der skabes et deploybart artefakt (f.eks. container-image eller binær fil).	• Hent kode fra versionsstyring (Git) • Installer afhængigheder (nugets) • Kør build-scripts (fx dotnet, npm, Docker build) • Publicér artefakter til repository (ACR, DockerHub)	Udviklere DevOps
TEST At sikre kvaliteten af buildet gennem automatiserede tests.	• Kør unit-, integration- og end-to-end-tests • Mål code coverage og statisk analyse • Brug testdata og mock-services • Rapporter resultater til CI-system	Udviklere QA CI-pipeline
STAGING At efterligne produktionen og validere hele systemet, inden release.	• Deployment i miljø identisk med produktion • Kør brugeraccepttest (UAT) og manuelle tests • Udfør performance- og sikkerhedstests • Godkend release til produktion	QA Produktansvarlige DevOps
PRODUCTION At levere den stabile, godkendte version af systemet til slutbrugerne.	• Deploy godkendt build • Overvågning, logging og alarmering • Backup og rollback-planer • Incident- og change management	Drift DevOps SRE

Deployment Diagrammer

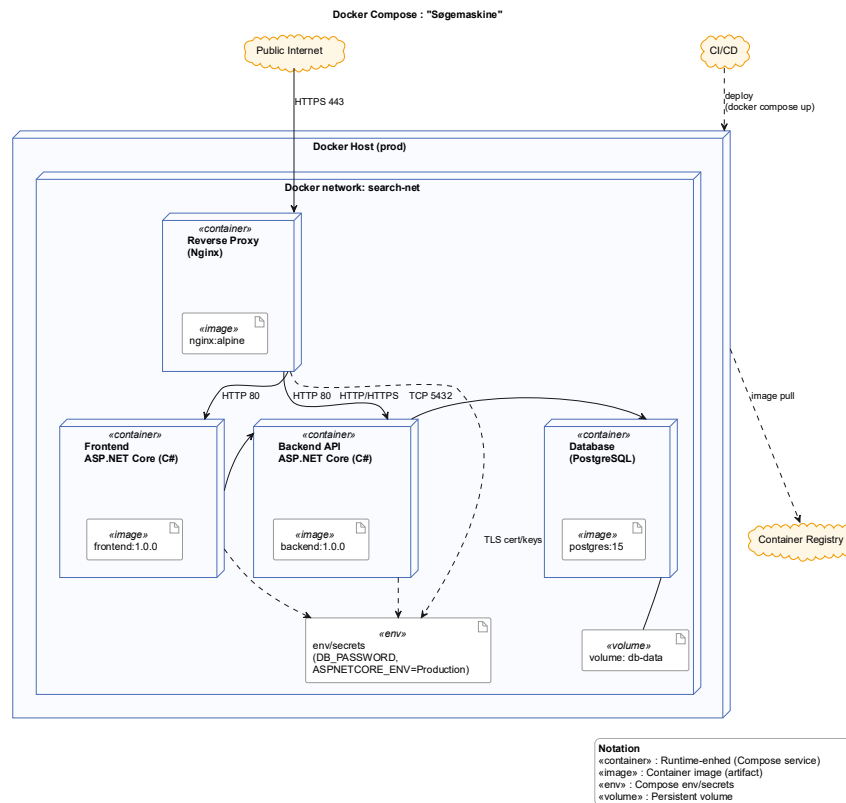


Diagrammer hjælper os med at forstå, kommunikere og dokumentere, hvordan et IT-system er bygget og fungerer.



- Skaber fælles forståelse mellem **udviklere**, **drift** og **forretning**
- Understøtter on-boarding, fejlsøgning og sikkerhedsgennemgang
- Viser sammenhæng mellem **komponenter**, **data** og **teknologi**
- Er levende dokumentation – ikke kun tegninger

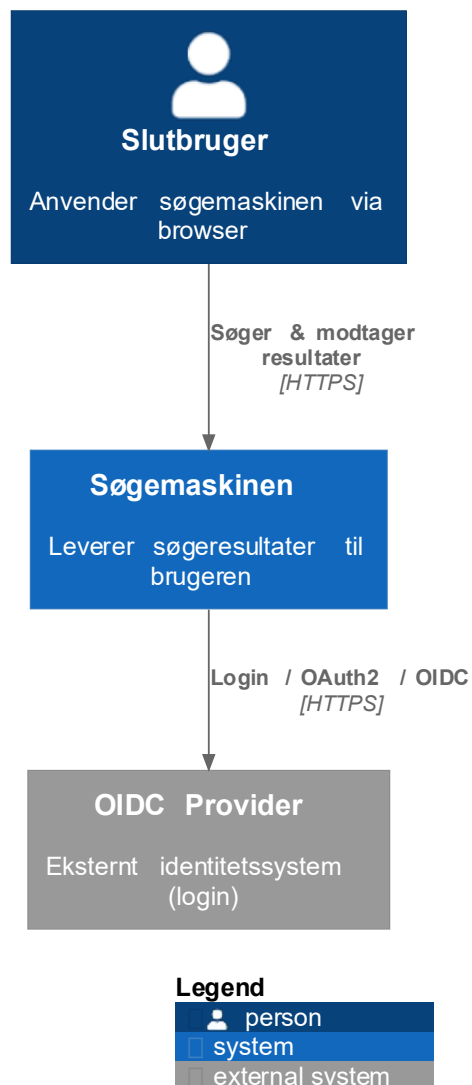
Et UML Deployment Diagram viser, hvordan softwarekomponenter (artefakter) udrulles på hardware- eller cloud-noder.



- Viser fysiske **noder** (servere, containere, cloud services)
- Viser **komponenter**/deployment units og forbindelser mellem dem
- Bruges til at dokumentere **infrastruktur** og **runtime-miljøer**
- Hjælper **DevOps** og **arkitekter** med at forstå driftssammenhænge

C4 – Context Diagram

C4 - Context: "Søgemaskinen"

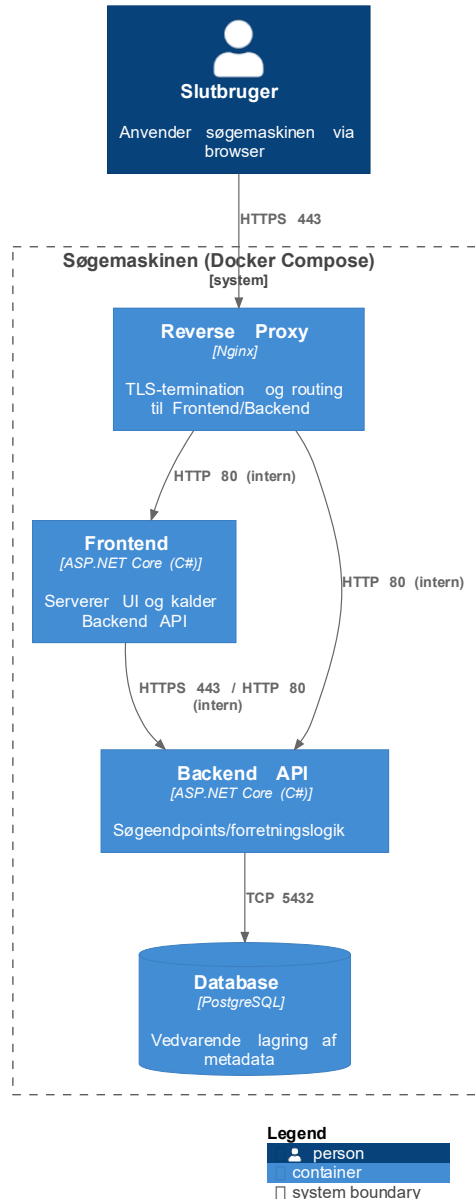


Et C4 Context Diagram viser systemet i sin kontekst – **hvem** bruger det, og **hvilke** andre systemer det interagerer med.

- **Fokus:** Systemets relation til omgivelserne
- Elementer: **Personer** (aktører), **software systemer** og **relationer**
- Viser det '**store billede**' uden tekniske detaljer
- Anvendes til at afgrænse systemets rolle i organisationen

C4 – Container Diagram

C4 - Container: "Søgemaskinen" (Docker Compose, minimal)



Viser systemets runtime-enheder – hvordan systemet er opdelt i containere eller **installationsbare enheder**.

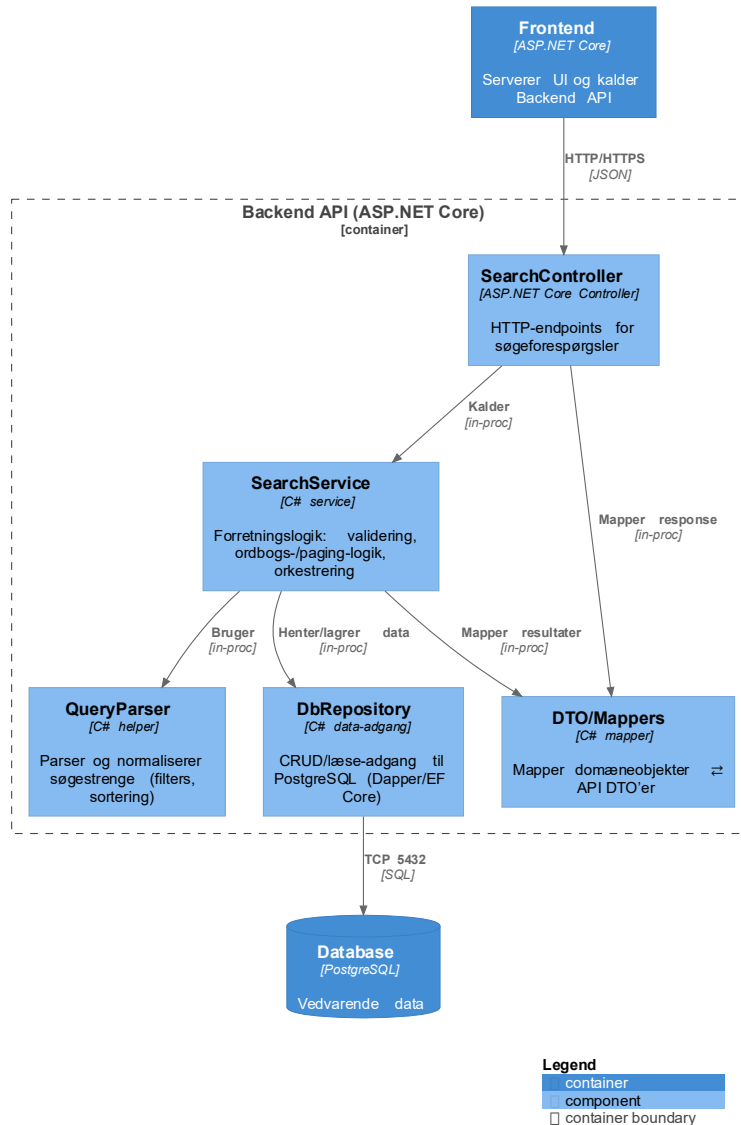
- Fokus: Installationsbare komponenter:
f.eks. → *API, frontend, database, message broker*
- Viser **teknologistak** og **forbindelser** mellem containere
- Hjælper med at forstå *drift, netværk* og *ansvar*
- Typisk næste niveau efter **Context**
– vis systemets arkitektur detaljer

C4 – Component Diagram

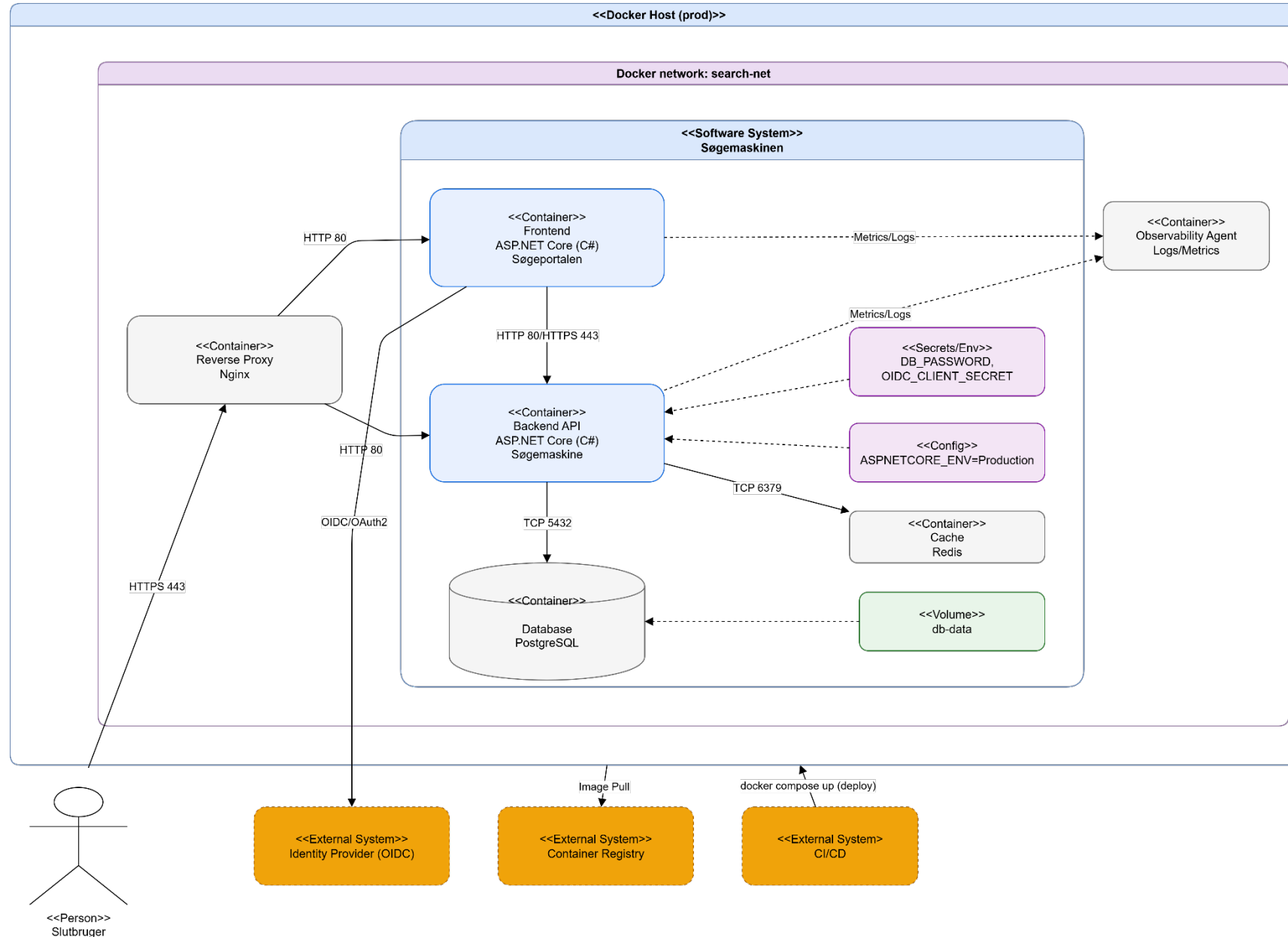
C4 - Component: Backend API i "Søgemaskinen" (minimal)

viser de interne softwarekomponenter inden for én container eller applikation.

- **Fokus:** Moduler, services, controllers, repositories mv.
- **Forklarer** hvordan koden er struktureret internt
- Bruges til at **forstå** afhængigheder og ansvar i applikationen
- **Nyttigt** for udviklere og arkitekter ved ændringer i koden



Søgemaskine: C4 - Container



Opgave



C4 Container diagram:

Søgemaskinen